Name: Farhan Mohamed Ibrahim
Date: 22 September 2026
Course/Class Title: Practical Development Setup: Flutter, Python, MySQL, and VS Code Integration.

Section 1: Task 1 - Flutter & Dart Setup
Flutter verification

Doctor summary (to see all details, run flutter doctor -v):
[√] Flutter (Channel stable, 3.47.4, on Microsoft Windows [Version 10.0.22621.4317], locale en-KE)
[√] Windows Version (11 Pro 64-bit, 22H2, 2009)
[√] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[√] Chrome - develop for the web
[√] Visual Studio - develop Windows apps (Visual Studio Community 2026 18.10.1)
[√] Connected device (3 available)
[√] Network resources

• No issues found!

Flutter target devices

C:\Users\user\my_first_app>flutter devices
Found 3 connected devices:
  Windows (desktop) • windows • windows-x64   • Microsoft Windows [Version 10.0.22621.4317]
  Chrome (web)   • chrome • web-javascript • Google Chrome 153.0.8010.48
  Edge (web)   • edge   • web-javascript • Microsoft Edge 153.0.4234.48

Run "flutter emulators" to list and start any available device emulators.

If you expected another device to be detected, please run "flutter doctor" to diagnose potential issues. You may also try increasing
the time to wait for connected devices with the "--device-timeout" flag. Visit https://flutter.dev/setup/ for troubleshooting tips.

HOT RELOAD AND HOT RESTART

Hot Reload allows a developer to apply code changes to a running Flutter application without completely restarting the application. It generally preserves the current application state, making it useful for quickly testing changes to the user interface, layout, styling, or other parts of the code.

Hot Restart completely restarts the Flutter application and clears the current application state. It is useful when changes require the application to start from the beginning or when Hot Reload does not properly reflect the changes.

In simple terms, Hot Reload is used to quickly see code changes while keeping the current state, whereas Hot Restart starts the application again from scratch.

Section 2: Task 2 - MySQL Database Management

1. SQL Script

The commands used are:

```
CREATE DATABASE school;


USE school;


CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(150) UNIQUE,
    enrolled_on DATE
);
INSERT INTO students (name, email, enrolled_on)
VALUES
    ('Ahmed Hassan', 'ahmed.hassan@example.com', '2026-09-22'),
    ('Amina Yusuf', 'amina.yusuf@example.com', '2026-09-22');
SELECT * FROM students;
```

The Results received:

```
mysql> SELECT * FROM students;
+----+--------------+-------------------------+-------------+
| id | name         | email                   | enrolled_on |
+----+--------------+-------------------------+-------------+
|  1 | Ahmed Hassan | ahmed.hassan@example.com | 2026-09-22  |
|  2 | Amina Yusuf  | amina.yusuf@example.com  | 2026-09-22  |
+----+--------------+-------------------------+-------------+
2 rows in set (0.00 sec)
```

Security Reflection

Using the MySQL `root` account for a direct application backend connection is poor security practice because `root` has extensive privileges over the database server. If the application's credentials were exposed or compromised, an attacker could potentially access, modify, or delete data beyond what the application actually needs.

A better approach is to create a dedicated application account and follow the principle of least privilege. For this project, I created a user called `school_app` and granted it only `SELECT`, `INSERT`, `UPDATE`, and `DELETE` privileges on the `school` database.

The SQL commands used were:

CREATE USER 'school_app'@'localhost' IDENTIFIED BY 'SchoolApp@2026!';

GRANT SELECT, INSERT, UPDATE, DELETE

ON school.* TO 'school_app'@'localhost';

SHOW GRANTS FOR 'school_app'@'localhost';

The `SHOW GRANTS` command was used to verify that the account received only the intended permissions on the `school` database.

Section 3: Task 3 - Python Virtual Environment

Python Environment and Dependency Isolation

A separate Python virtual environment was created to keep the project's dependencies isolated from the system-wide Python installation. The `requests` package was installed inside the virtual environment and the installed packages were verified using `pip list`. Finally, the environment was exported to a `requirements.txt` file.

Commands Used

mkdir python_setup_lab

cd python_setup_lab

python -m venv venv

venv\Scripts\activate

pip install requests

pip list

pip freeze > requirements.txt

Verification

The virtual environment was successfully activated, as shown by the `(venv)` prefix in the terminal. The `pip list` command confirmed that the `requests` package and its dependencies were installed successfully. The `pip freeze > requirements.txt` command successfully created the `requirements.txt` file inside the `python_setup_lab` directory.

Section 4: Task 4 - VS Code Workspace Screenshot

VS Code was configured with the required Flutter, Dart, Python, Pylance, and MySQL extensions. The Python virtual environment (`venv`) was activated and configured for the `python_setup_lab` project.

The screenshot below shows the installed extensions and the active Python virtual environment in the integrated terminal.wait

